READ DATA

```
import os
import random
from google.colab import drive

drive.mount('/content/drive')
train_pneumonia_dir = '/content/drive/MyDrive/TA/Dataset/train/PNEUMONIA'
test_pneumonia_dir = '/content/drive/MyDrive/TA/Dataset/test/PNEUMONIA'
train_normal_dir = '/content/drive/MyDrive/TA/Dataset/train/NORMAL'
test_normal_dir = '/content/drive/MyDrive/TA/Dataset/test/NORMAL'
```

```
Mounted at /content/drive
```

```
def get_all_files(dirs, allowed_extensions=['.jpg', '.jpeg', '.png']):
    all_files = []
    for dir_path in dirs:
        files = [os.path.join(dir_path, f) for f in os.listdir(dir_path)
                 if not f.startswith('.') and os.path.splitext(f)[1].lower() in allowed_extensions]
        all_files.extend(files)
    return all_files

def random_select_files(file_list, subset_size):
    if subset_size > len(file_list):
        raise ValueError("Jumlah file yang diminta lebih besar daripada jumlah file yang tersedia.")
    return random.sample(file_list, subset_size)

def count_files(directory):
  return len(os.listdir(directory))
```

```
all_pneumonia_files = get_all_files([train_pneumonia_dir, test_pneumonia_dir])
all_normal_files = get_all_files([train_normal_dir, test_normal_dir])

print(f"train pneumon : {count_files(train_pneumonia_dir)}")
print(f"test pneumon : {count_files(test_pneumonia_dir)}")
print(f"train norm : {count_files(train_normal_dir)}")
print(f"test norm : {count_files(test_normal_dir)}")

print(f"pneumon : {len(all_pneumonia_files)}")
```

```
print(f"norm : {len(all_normal_files)}")
```

```
train pneumon : 3875
test pneumon : 390
train norm : 1341
test norm : 234
pneumon : 4265
norm : 1575
```

BALANCING

```
train_pneumonia_count = 3200
test_pneumonia_count = 1065
train_normal_count = 3200
test_normal_count = 1065

test_pneumonia_samples = random.sample(all_pneumonia_files, test_pneumonia_count)
test_normal_samples = random.sample(all_normal_files, test_normal_count)

pneumonia_train_pool = [f for f in all_pneumonia_files if f not in test_pneumonia_samples]
normal_train_pool = [f for f in all_normal_files if f not in test_normal_samples]

train_pneumonia_samples = random.choices(pneumonia_train_pool, k=train_pneumonia_count)
train_normal_samples = random.choices(normal_train_pool, k=train_normal_count)
```

```
print(f"train_pneumonia_samples : {len(train_pneumonia_samples)}")
print(f"test_pneumonia_samples : {len(test_pneumonia_samples)}")
print(f"train_normal_samples : {len(train_normal_samples)}")
print(f"test_normal_samples : {len(test_normal_samples)}")
```

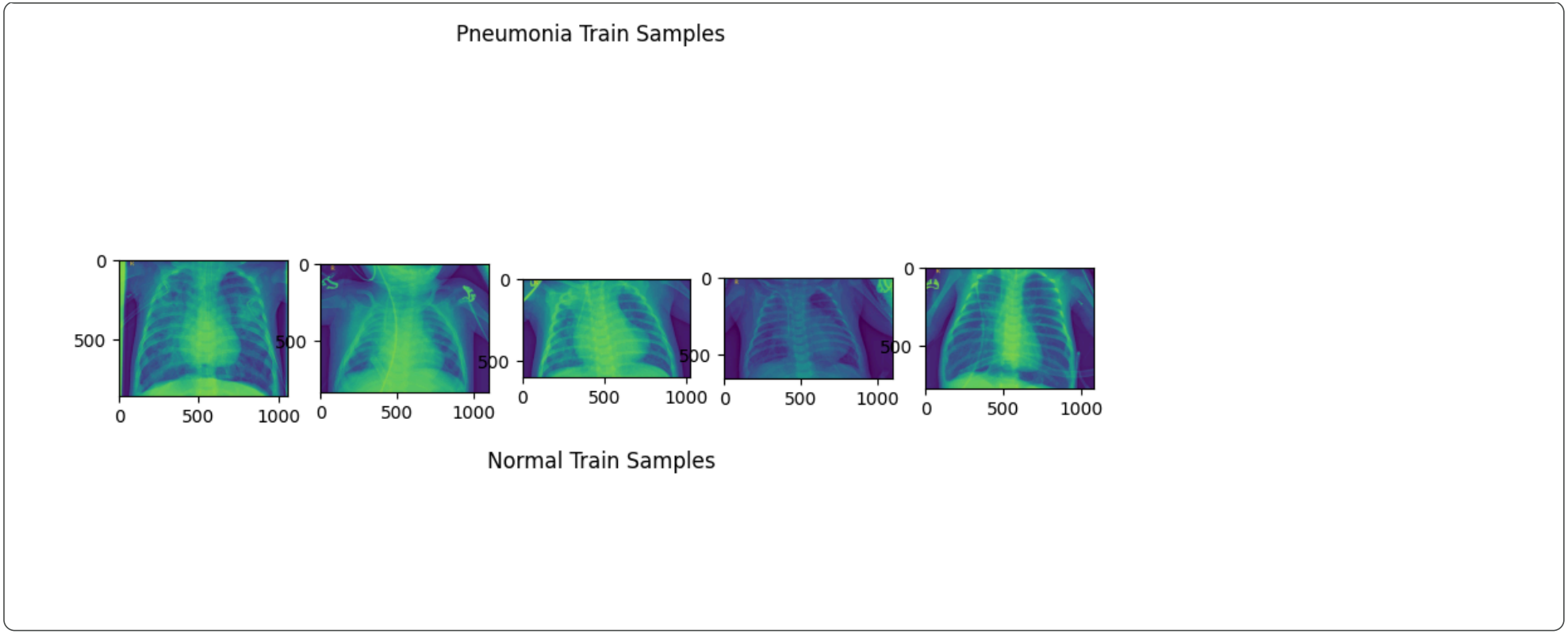
```
train_pneumonia_samples : 3200
test_pneumonia_samples : 1065
train_normal_samples : 3200
test_normal_samples : 1065
```

```
from PIL import Image
import matplotlib.pyplot as plt

def show_sample_images(sample_files,ammount, title):
    plt.figure(figsize=(10, 5))
    plt.suptitle(title)
    for i in range(ammount):
        plt.subplot(1, ammount, i+1)
```

```
            img = Image.open(sample_files[i])
            plt.imshow(img)
        plt.show()

    show_sample_images(train_pneumonia_samples, 5, 'Pneumonia Train Samples')
    show_sample_images(train_normal_samples, 5, 'Normal Train Samples')
    show_sample_images(test_pneumonia_samples, 5, 'Pneumonia Test Samples')
    show_sample_images(test_normal_samples, 5, 'Normal Test Samples')
```



RESIZE

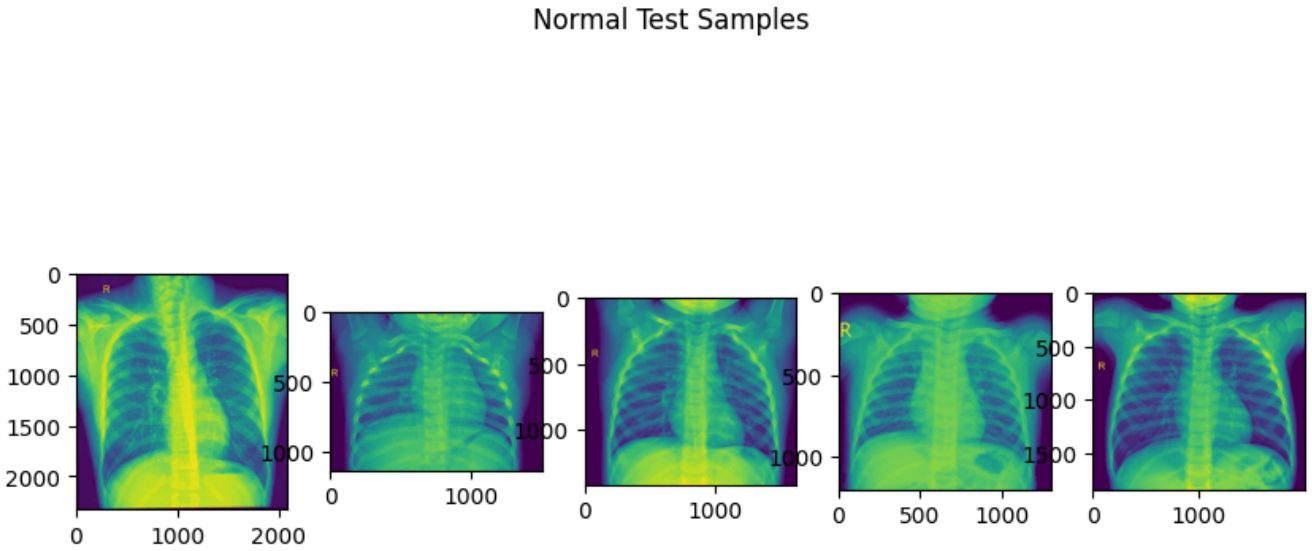
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
import numpy as np
from tensorflow.keras.preprocessing.image import load_img, img_to_array

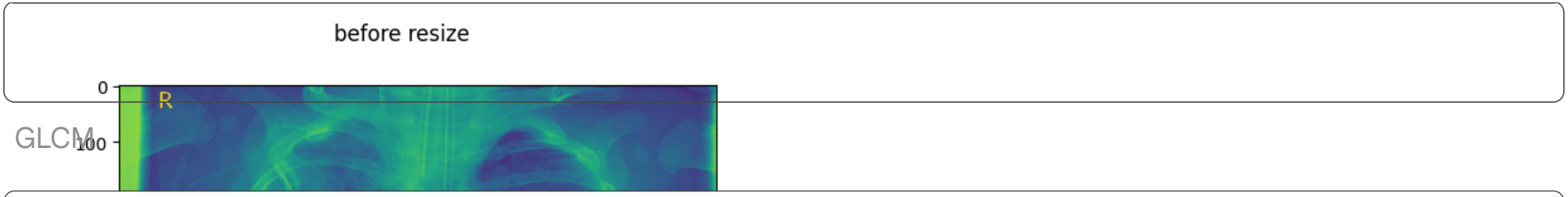
def load_images(file_paths, target_size=(224, 224)):
    images = []
    for i, file_path in enumerate(file_paths):
        img = load_img(file_path, target_size=target_size)
        img_array = img_to_array(img)
        img_array = img_array - 30
        img_array = img_array / 255.0
        images.append(img_array)
    return images

train_pneumonia_images = load_images(train_pneumonia_samples)
test_pneumonia_images = load_images(test_pneumonia_samples)
```

```
train_normal_images = load_images(train_normal_samples)
test_normal_images = load_images(test_normal_samples)
```

```
show_sample_images(train_pneumonia_samples, 1, 'before resize')
plt.imshow(train_pneumonia_images[0])
plt.title('After Resizing')
```





GLCM

```
import numpy as np
import cv2
from skimage.feature import graycomatrix, graycoprops

def calculate_glcm(image):
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    distances = [1, 2, 3]
    angles = [0, np.pi/4, np.pi/2, 3*np.pi/4]

    glcm = graycomatrix(gray_image,
                        distances=distances,
                        angles=angles,
                        levels=256,
                        symmetric=True,
                        normed=True)

    properties = ['contrast', 'dissimilarity', 'homogeneity', 'energy', 'correlation', 'ASM']
    features = {}

    for prop in properties:
        values = graycoprops(glcm, prop).flatten()
        features[f'{prop}_mean'] = np.mean(values)
        features[f'{prop}_std'] = np.std(values)
        features[f'{prop}_var'] = np.var(values)

    glcm_entropy = -np.sum(glcm * np.log2(glcm + 1e-10))
    features['entropy'] = glcm_entropy

    return features
```



```
import cv2

def extract_glcm_features(image_list):
    glcm_features_list = []
    for image in image_list:
        image_uint8 = (image * 255).astype(np.uint8)
        features = calculate_glcm(image_uint8)
```

```
            glcm_features_list.append(features)
        return glcm_features_list

    train_pneumonia_glcm = extract_glcm_features(train_pneumonia_images)
    test_pneumonia_glcm = extract_glcm_features(test_pneumonia_images)
    train_normal_glcm = extract_glcm_features(train_normal_images)
    test_normal_glcm = extract_glcm_features(test_normal_images)

    print("GLCM features for the first pneumonia training image:")
    train_pneumonia_glcm[0]
```

```
GLCM features for the first pneumonia training image:
{'contrast_mean': np.float64(1215.7637282097342),
 'contrast_std': np.float64(604.2551625381589),
 'contrast_var': np.float64(365124.30145401676),
 'dissimilarity_mean': np.float64(13.446299555270656),
 'dissimilarity_std': np.float64(4.462842562958955),
 'dissimilarity_var': np.float64(19.916963741758057),
 'homogeneity_mean': np.float64(0.16560993036757773),
 'homogeneity_std': np.float64(0.04336721745535526),
 'homogeneity_var': np.float64(0.0018807155498200701),
 'energy_mean': np.float64(0.014852845892011206),
 'energy_std': np.float64(0.001976914214063014),
 'energy_var': np.float64(3.908189809764385e-06),
 'correlation_mean': np.float64(0.8109860402664245),
 'correlation_std': np.float64(0.09408777855407914),
 'correlation_var': np.float64(0.008852510073241435),
 'ASM_mean': np.float64(0.00022451522090159858),
 'ASM_std': np.float64(6.113779159230574e-05),
 'ASM_var': np.float64(3.7378295607842105e-09),
 'entropy': np.float64(153.51413650128237)}
```

```
#import pandas as pd
#from google.colab import files

#train_pneumonia_df = pd.DataFrame(train_pneumonia_glcm)
#train_pneumonia_df['label'] = 'PNEUMONIA'

#test_pneumonia_df = pd.DataFrame(test_pneumonia_glcm)
#test_pneumonia_df['label'] = 'PNEUMONIA'

#train_normal_df = pd.DataFrame(train_normal_glcm)
#train_normal_df['label'] = 'NORMAL'

#test_normal_df = pd.DataFrame(test_normal_glcm)
#test_normal_df['label'] = 'NORMAL'
```

```
#all_glcm_df = pd.concat([train_pneumonia_df, test_pneumonia_df, train_normal_df, test_normal_df], ignore_index=True)

#excel_filename = 'glcm_features.xlsx'
#all_glcm_df.to_excel(excel_filename, index=False)

#files.download(excel_filename)

#print(f"GLCM features saved to '{excel_filename}' and downloaded.")
```

CNN

```
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.regularizers import l2

# GLCM - DF
train_pneumonia_glcm_df = pd.DataFrame(train_pneumonia_glcm)
test_pneumonia_glcm_df = pd.DataFrame(test_pneumonia_glcm)
train_normal_glcm_df = pd.DataFrame(train_normal_glcm)
test_normal_glcm_df = pd.DataFrame(test_normal_glcm)

train_pneumonia_glcm_df['label'] = 1  # 1 for pneumonia
test_pneumonia_glcm_df['label'] = 1
train_normal_glcm_df['label'] = 0  # 0 for normal
test_normal_glcm_df['label'] = 0

train_glcm_df = pd.concat([train_pneumonia_glcm_df, train_normal_glcm_df], ignore_index=True)
test_glcm_df = pd.concat([test_pneumonia_glcm_df, test_normal_glcm_df], ignore_index=True)


# Separate features
X_train = train_glcm_df.drop('label', axis=1)
y_train = train_glcm_df['label']
X_test = test_glcm_df.drop('label', axis=1)
y_test = test_glcm_df['label']

# Normal
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# Model
model = Sequential([
    Dense(128, activation='relu', kernel_regularizer=l2(0.0001), input_shape=(X_train.shape[1],)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(64, activation='relu', kernel_regularizer=l2(0.0001)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(32, activation='relu', kernel_regularizer=l2(0.0001)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])

optimizer = Adam(learning_rate=0.001)
class_weights = {0: 1.5, 1: 0.8}

callbacks = [
    EarlyStopping(monitor='val_loss', patience=50, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=25, min_lr=1e-6)
]

model.compile(optimizer=optimizer,
              loss='binary_crossentropy',
              metrics=['accuracy', tf.keras.metrics.AUC()])

model.summary()

history = model.fit(
    X_train, y_train,
    epochs=50,
    #epochs=100,
    batch_size=32,
    validation_split=0.3,
    callbacks=None,
    #callbacks=callbacks,
    class_weight=class_weights,
)
```



```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_6"
```

Layer (type)	Output Shape	Param #
dense_18 (Dense)	(None, 128)	2,560
batch_normalization_12 (BatchNormalization)	(None, 128)	512
dropout_12 (Dropout)	(None, 128)	0
dense_19 (Dense)	(None, 64)	8,256
batch_normalization_13 (BatchNormalization)	(None, 64)	256
dropout_13 (Dropout)	(None, 64)	0
dense_20 (Dense)	(None, 32)	2,080
batch_normalization_14 (BatchNormalization)	(None, 32)	128
dropout_14 (Dropout)	(None, 32)	0
dense_21 (Dense)	(None, 1)	33

```
 Total params: 13,825 (54.00 KB)
 Trainable params: 13,377 (52.25 KB)
 Non-trainable params: 448 (1.75 KB)
Epoch 1/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 12s 16ms/step - accuracy: 0.7554 - auc_6: 0.8483 - loss: 0.5444 - val_accuracy: 0.6057 - val_auc_6: 0.0000e+00 - val_loss: 0.74
Epoch 2/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - accuracy: 0.8406 - auc_6: 0.9059 - loss: 0.4168 - val_accuracy: 0.8104 - val_auc_6: 0.0000e+00 - val_loss: 0.49
Epoch 3/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8533 - auc_6: 0.9174 - loss: 0.3846 - val_accuracy: 0.8734 - val_auc_6: 0.0000e+00 - val_loss: 0.383
Epoch 4/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8638 - auc_6: 0.9228 - loss: 0.3697 - val_accuracy: 0.8969 - val_auc_6: 0.0000e+00 - val_loss: 0.335
Epoch 5/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8710 - auc_6: 0.9290 - loss: 0.3538 - val_accuracy: 0.9099 - val_auc_6: 0.0000e+00 - val_loss: 0.276
Epoch 6/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8721 - auc_6: 0.9312 - loss: 0.3481 - val_accuracy: 0.9099 - val_auc_6: 0.0000e+00 - val_loss: 0.274
Epoch 7/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8723 - auc_6: 0.9360 - loss: 0.3356 - val_accuracy: 0.8932 - val_auc_6: 0.0000e+00 - val_loss: 0.319
Epoch 8/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8857 - auc_6: 0.9431 - loss: 0.3149 - val_accuracy: 0.9161 - val_auc_6: 0.0000e+00 - val_loss: 0.288
Epoch 9/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8701 - auc_6: 0.9369 - loss: 0.3319 - val_accuracy: 0.8953 - val_auc_6: 0.0000e+00 - val_loss: 0.310
Epoch 10/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8862 - auc_6: 0.9451 - loss: 0.3097 - val_accuracy: 0.9052 - val_auc_6: 0.0000e+00 - val_loss: 0.307
Epoch 11/50
```

```
Epoch 11/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8875 - auc_6: 0.9465 - loss: 0.3066 - val_accuracy: 0.9000 - val_auc_6: 0.0000e+00 - val_loss: 0.3068
Epoch 12/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8902 - auc_6: 0.9514 - loss: 0.2928 - val_accuracy: 0.9198 - val_auc_6: 0.0000e+00 - val_loss: 0.268
Epoch 13/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8884 - auc_6: 0.9518 - loss: 0.2916 - val_accuracy: 0.9266 - val_auc_6: 0.0000e+00 - val_loss: 0.283
Epoch 14/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8888 - auc_6: 0.9487 - loss: 0.2999 - val_accuracy: 0.9307 - val_auc_6: 0.0000e+00 - val_loss: 0.259
Epoch 15/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8891 - auc_6: 0.9512 - loss: 0.2917 - val_accuracy: 0.9182 - val_auc_6: 0.0000e+00 - val_loss: 0.2904
Epoch 16/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8958 - auc_6: 0.9549 - loss: 0.2813 - val_accuracy: 0.9073 - val_auc_6: 0.0000e+00 - val_loss: 0.262
Epoch 17/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8884 - auc_6: 0.9518 - loss: 0.2911 - val_accuracy: 0.9187 - val_auc_6: 0.0000e+00 - val_loss: 0.260
Epoch 18/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8951 - auc_6: 0.9531 - loss: 0.2881 - val_accuracy: 0.9401 - val_auc_6: 0.0000e+00 - val_loss: 0.249
Epoch 19/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8882 - auc_6: 0.9553 - loss: 0.2803 - val_accuracy: 0.9130 - val_auc_6: 0.0000e+00 - val_loss: 0.3019
Epoch 20/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8964 - auc_6: 0.9550 - loss: 0.2809 - val_accuracy: 0.9005 - val_auc_6: 0.0000e+00 - val_loss: 0.2839
Epoch 21/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8982 - auc_6: 0.9595 - loss: 0.2663 - val_accuracy: 0.9062 - val_auc_6: 0.0000e+00 - val_loss: 0.2878
Epoch 22/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8964 - auc_6: 0.9557 - loss: 0.2785 - val_accuracy: 0.9187 - val_auc_6: 0.0000e+00 - val_loss: 0.276
Epoch 23/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8955 - auc_6: 0.9573 - loss: 0.2738 - val_accuracy: 0.9344 - val_auc_6: 0.0000e+00 - val_loss: 0.2569
Epoch 24/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8971 - auc_6: 0.9539 - loss: 0.2835 - val_accuracy: 0.9214 - val_auc_6: 0.0000e+00 - val_loss: 0.269
Epoch 25/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8955 - auc_6: 0.9563 - loss: 0.2779 - val_accuracy: 0.9250 - val_auc_6: 0.0000e+00 - val_loss: 0.265
Epoch 26/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9009 - auc_6: 0.9580 - loss: 0.2709 - val_accuracy: 0.9130 - val_auc_6: 0.0000e+00 - val_loss: 0.283
Epoch 27/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9004 - auc_6: 0.9589 - loss: 0.2696 - val_accuracy: 0.9333 - val_auc_6: 0.0000e+00 - val_loss: 0.256
Epoch 28/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9002 - auc_6: 0.9599 - loss: 0.2628 - val_accuracy: 0.9240 - val_auc_6: 0.0000e+00 - val_loss: 0.252
Epoch 29/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9076 - auc_6: 0.9627 - loss: 0.2555 - val_accuracy: 0.9187 - val_auc_6: 0.0000e+00 - val_loss: 0.2609
Epoch 30/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9018 - auc_6: 0.9587 - loss: 0.2688 - val_accuracy: 0.9255 - val_auc_6: 0.0000e+00 - val_loss: 0.270
Epoch 31/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8996 - auc_6: 0.9597 - loss: 0.2651 - val_accuracy: 0.9094 - val_auc_6: 0.0000e+00 - val_loss: 0.282
Epoch 32/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8982 - auc_6: 0.9601 - loss: 0.2645 - val_accuracy: 0.9323 - val_auc_6: 0.0000e+00 - val_loss: 0.274
Epoch 33/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9051 - auc_6: 0.9605 - loss: 0.2616 - val_accuracy: 0.9281 - val_auc_6: 0.0000e+00 - val_loss: 0.283
Epoch 34/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9042 - auc_6: 0.9589 - loss: 0.2643 - val_accuracy: 0.9255 - val_auc_6: 0.0000e+00 - val_loss: 0.263
Epoch 35/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9016 - auc_6: 0.9618 - loss: 0.2578 - val_accuracy: 0.9380 - val_auc_6: 0.0000e+00 - val_loss: 0.257
Epoch 36/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.9049 - auc_6: 0.9616 - loss: 0.2575 - val_accuracy: 0.9417 - val_auc_6: 0.0000e+00 - val_loss: 0.251
Epoch 37/50
```

```
from sklearn.metrics import precision_score, recall_score
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
test_loss, test_accuracy, test_auc = model.evaluate(X_test, y_test, verbose=0)
test_precision = precision_score(y_test, y_pred_classes)
test_recall = recall_score(y_test, y_pred_classes)

print(f"- Loss: {test_loss:.4f}")
print(f"- AUC: {test_auc:.4f}")
print(f"- Accuracy: {test_accuracy:.4f}")
```

```
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9069 - auc_6: 0.9625 - loss: 0.2541 - val_accuracy: 0.9318 - val_auc_6: 0.0000e+00 - val_loss: 0.2480
67/67 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
Epoch 44/50
- Loss: 0.3124
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9025 - auc_6: 0.9602 - loss: 0.2612 - val_accuracy: 0.9453 - val_auc_6: 0.0000e+00 - val_loss: 0.229
- AUC: 0.9560
Epoch 45/50
- Accuracy: 0.8901
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9036 - auc_6: 0.9619 - loss: 0.2572 - val_accuracy: 0.9391 - val_auc_6: 0.0000e+00 - val_loss: 0.2217
```

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_classes))
```

```
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9132 - auc_6: 0.9658 - loss: 0.2398 - val_accuracy: 0.9380 - val_auc_6: 0.0000e+00 - val_loss: 0.2607
Epoch 49/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9096 - auc_6: 0.9650 - loss: 0.2471 - val_accuracy: 0.9422 - val_auc_6: 0.0000e+00 - val_loss: 0.2184
Epoch 50/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9094 - auc_6: 0.9662 - loss: 0.2430 - val_accuracy: 0.9302 - val_auc_6: 0.0000e+00 - val_loss: 0.2438
              precision    recall  f1-score   support

           0       0.91      0.88      0.89      1065
           1       0.87      0.92      0.89      1065

    accuracy                           0.89      2130
   macro avg       0.89      0.89      0.89      2130
weighted avg       0.89      0.89      0.89      2130
```

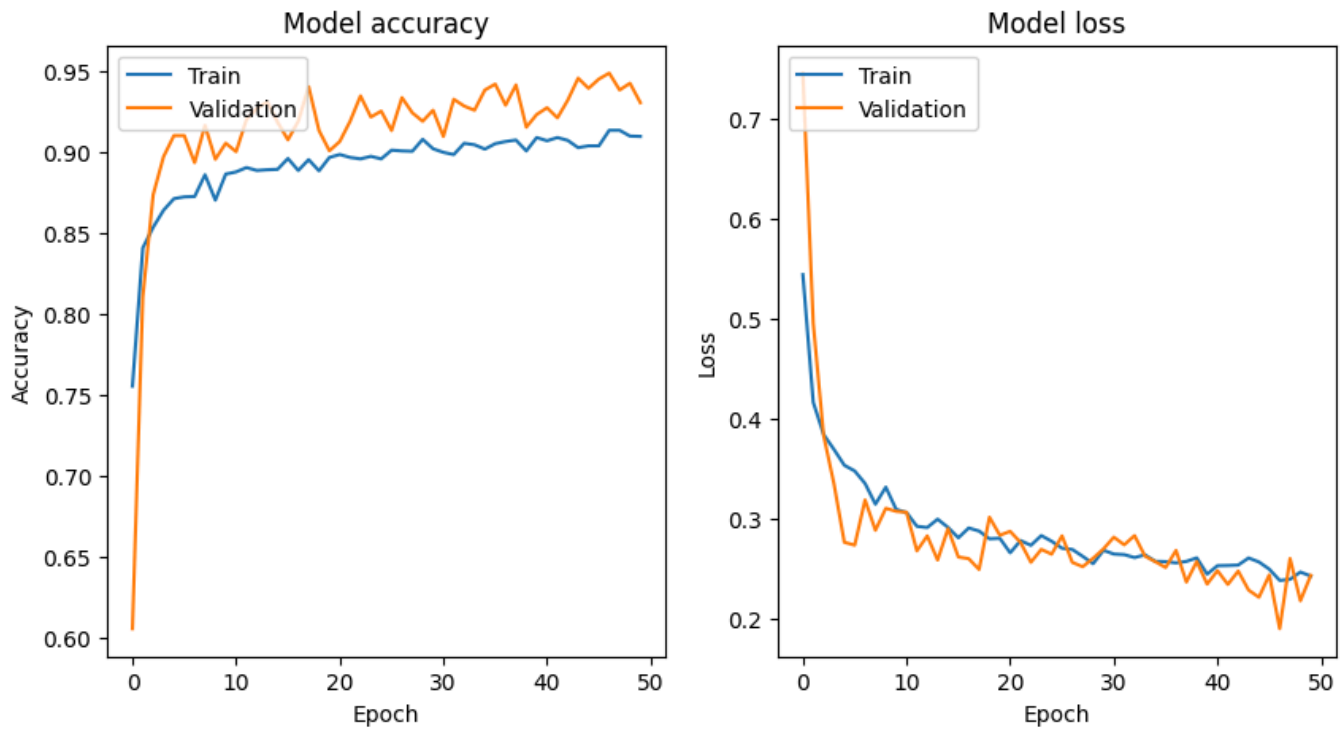
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
```

```
plt.legend(['Train', 'Validation'], loc='upper left')

plt.show()
```



PREDICT

```
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
```

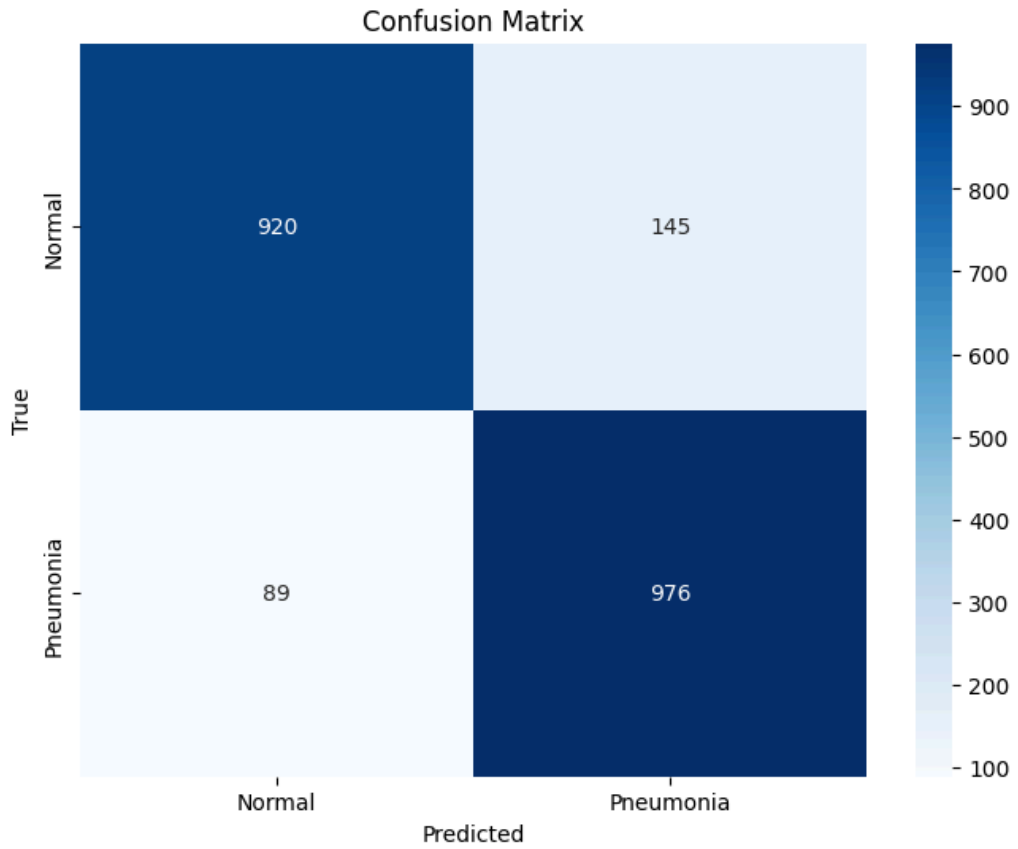
```
67/67 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

conf_matrix = confusion_matrix(y_test, y_pred_classes)

plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
```

```
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```




```
model.save("Pnem_sim.h5")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We re
```

```
from google.colab import files
files.download("Pnem_sim.h5")
```

```
import pickle

with open('scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)
```

```
files.download("scaler.pkl")
```

CODE END HERE

Tes SVM

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Create and train the SVM model
svm_model = SVC(kernel='linear', random_state=42) # You can choose different kernels
svm_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_svm = svm_model.predict(X_test)

# Evaluate the SVM model
print("\n--- SVM Model Evaluation ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_svm):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_svm))

# Confusion Matrix
conf_matrix_svm = confusion_matrix(y_test, y_pred_svm)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_svm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix (SVM)')
plt.show()
```

```
--- SVM Model Evaluation ---
Accuracy: 0.8864

Classification Report:
              precision    recall  f1-score   support

           0       0.87      0.91      0.89      1065
           1       0.90      0.87      0.88      1065

    accuracy                           0.89      2130
   macro avg       0.89      0.89      0.89      2130
weighted avg       0.89      0.89      0.89      2130
```

